CRASH COURSE #2 · THURSDAY 22 OCTOBER 2026 · 17:00 → 19:00 · ON ZOOM

Vibe coding & AI-assisted development

Direct the agent. Check the result.

Where AI agents come from, how they change research, and how to work with one: hands on, in pairs, on real data.

FacMed Hack #1 · Digital Phenotyping of Mental Health · Fri 6 – Sat 7 November 2026 · Campus Biotech · in English

The road to 6 November

Thu 8 Oct · Crash Course #1 · Digital phenotyping & the Swiss data rules
Today · Crash Course #2 · Vibe coding & AI-assisted development
Thu 29 Oct · Crash Course #3 · Grant writing for pop-up projects, with the RGO
Fri 6 – Sat 7 Nov · the hackathon, 32 hours

Université de Genève · Faculty of Medicine · Université de Genève · Faculty of Medicine · Synapsy centre for neuroscience and mental health research · Wyss Center · neuro @campusbiotech · HUG Hôpitaux Universitaires Genève · EPFL · Université de Genève · Swiss Center for Affective Sciences · Sparkboard

> **"AI is obviously going to have a massive impact on how we do science, and it's hard to know what the long run looks like."**

Anthony Zador · Cold Spring Harbor Laboratory · *The Transmitter*, June 2026

3 min

BEFORE ANYTHING ELSE · 90 SECONDS · ONE OR TWO LINES IN THE ZOOM CHAT

Where do you imagine vibe coding and agentic coding will change science? Name one place, and what changes there.

Your own field or any other. A method, an instrument, a job in the lab, a stage of publishing: be specific.

> *"What would you do if you had all the models of the brain you wanted?"*
>
> Grace Lindsay, NYU, responding to this week's pre-read (DePasquale, *The Transmitter*, June 2026)

TONIGHT · THREE OBJECTIVES

By 19:00 you should…

1 · IMAGINE

See your place in agentic science

Where these tools come from, what they already do in research, and what becomes yours to do: your contribution, your career, your purpose.

2 · UNDERSTAND

Know how vibe coding works

Think fast and slow. Try, fail and retry cheaply. Git as the safety net, run by the agent for you. Working as a team, with agents in the loop.

3 · PREPARE

Know how it runs at the hack

Whether you already have the tools or not, both paths are covered: your own setup, or the HPE / NVIDIA environment, open to every team to build and test on.

Built on the course *Vibe Coding for Scientists* (github.com/wazaahhh/vibe-coding-crash-course, CC BY-NC 4.0). Pre-reads: DePasquale (2026) and Schuck (2026), both in *The Transmitter*.

TONIGHT · 17:00 → 19:00

Seven parts, six polls along the way

TIME	PART	POLL	OBJ.
17:00	Opening: a quote, a question, three objectives	—	1
17:05	**1 · Where this comes from**: from artificial neurons to coding agents; uses; opportunities and pitfalls (pre-read 1)	1 · 2	1
17:25	**2 · How vibe coding works**: four disciplines, how to ask, three dials	—	2
17:33	**3 · Git without the jargon**: save points, and how the agent runs git for you	3	2
17:45	**4 · Practical in pairs** (breakout rooms): synthetic sleep data, then a real EEG recording	4	2
18:25	**5 · Review hunt**: would you sign this AI-written analysis?	5	2
18:33	**6 · The hack**: where we are, how it runs, with or without your own tools	—	3
18:43	**7 · A future worth building**: AI governance in science (pre-read 2), your place	6	1

20 min

17:05 · PART 1 · OBJECTIVE 1 · IMAGINE

Where this comes from

From an artificial neuron in 1943 to an agent that edits your analysis code. What these tools are, what scientists do with them, and where they fail.

A SHORT HISTORY · PART 1

It started in your field

YEAR	WHAT HAPPENED
1943	**McCulloch & Pitts** model a neuron as a logic unit: the first artificial neuron
1958	**Rosenblatt's perceptron**: a network of artificial neurons that learns from examples
1982	**Hopfield networks**: memories stored as stable states of a network
1986	**Backpropagation** popularised (Rumelhart, Hinton & Williams): how multi-layer networks learn
2012	**Deep learning** wins image recognition by a wide margin (AlexNet): GPUs + big data + depth
2024	**Nobel Prize in Physics** to John Hopfield and Geoffrey Hinton, for the foundations of machine learning with artificial neural networks

A SHORT HISTORY · PART 2

From predicting the next word to editing your code

YEAR	WHAT HAPPENED
2017	**The Transformer** ("Attention is all you need"): the architecture behind every modern language model
2020	**GPT-3**: 175 billion parameters; scale alone produces surprising abilities
2021	**GitHub Copilot** autocompletes code inside the editor
Nov 2022	**ChatGPT**: language models reach everyone, through a chat window
2023–24	Claude, GPT-4, open-weight models (Llama, Qwen…); **"reasoning" models** that work step by step before answering
2025	**Coding agents** (Claude Code, OpenAI Codex, Copilot agent mode) work inside your project; Andrej Karpathy names **"vibe coding"**
2026	Agents in research labs: model fitting, analysis pipelines, lab AI policies (this week's two pre-reads)

UNDER THE HOOD, IN PLAIN WORDS

What a large language model (LLM) actually is

1. A very large **neural network** trained to **predict the next word piece** ("token") on a huge share of the written web, books and code.
2. Then **tuned** with examples and human feedback to follow instructions and be helpful.
3. It holds no database of facts: it generates the most **plausible continuation** of what it has been given.

SO…	IN PRACTICE
Fluent ≠ true	It can invent a method, a number or a citation, in flawless prose
Training cut-off	It does not know what happened after its training data
Context window	It only "sees" what fits in its working memory: files, chat, instructions
Not deterministic	The same question can get different answers

MODULE 03 · TWO MODES

From chatbot to agent

CHAT (BROWSER, APP)	AGENT (EDITOR + YOUR PROJECT FOLDER)
You paste code, data, an error; it answers	It **reads your files, runs code, edits in place**
Zero setup	One-time setup
You carry the context, message after message	It sees the whole project
A quick question, a snippet	A real, multi-file analysis

An agent is an LLM in a loop with tools

You give a goal → it **plans** → it **acts** (reads a file, runs a script, searches) → it **looks at the result** or the error → it adjusts → … → it reports back.

You approve the plan, watch the steps, and decide what to keep.

Code is where agents work best, and where "vibe coding" was born

Why coding first

- Code is text, and there is a vast public corpus of it.
- Code can be **run**: errors and tests give the agent immediate feedback.
- The result can be **checked**, by the machine and by you.

"Vibe coding" (Andrej Karpathy, 2025)

- You describe **what you want** in plain language; the model writes the code.
- Tool-agnostic: Claude Code, Codex, Copilot, Cline, or paste-into-chat.

> The honest version for scientists: the vibe is in **how you produce** code, never in whether it's correct.

One sentence in, a working analysis out

- **Neuro / EEG**: *"Load this `.edf`, band-pass 8–12 Hz, plot alpha power per channel as a topomap."* → it wires up MNE, filters, renders the scalp map.
- **Genomics**: *"Here's my RNA-seq counts table: give me a volcano plot, label the top 10 genes by adjusted p-value."* → it reads the file, runs the statistics, returns a labelled figure **and** the code.
- **Climate / field data**: *"30 years of daily rainfall: fit a trend, test it, plot it with a 12-month rolling mean."*
- **Imaging**: *"Count the nuclei in every image in `data/dapi/`, one CSV row per file."* → it plans, writes, runs on 200 images. **You** notice the dense clusters look undercounted, ask for three overlays: touching nuclei had merged.

> The question stays yours. The **plumbing** stops being the bottleneck. Looking, doubting, correcting: that last step is the whole job.

HOW SCIENTISTS USE IT TODAY

Six everyday uses, and a few frontier ones

EVERYDAY	WHAT IT DOES FOR YOU
Literature	find, summarise, compare papers
Analysis code	cleaning, statistics, figures, pipelines
Modelling	from a word model to a fitted model in hours
Writing	editing, restructuring, translating
Learning	explaining unfamiliar code or methods
Tools	small apps, dashboards, lab utilities

At the frontier

AlphaFold predicts protein structures (Nobel Prize in Chemistry 2024) · **AI "co-scientists"** propose hypotheses (Google, 2025) · **automated research pipelines** draft whole studies (The AI Scientist, 2024), with mixed quality

AND IN NEUROSCIENCE

Already happening in our field

Predicting results

On a benchmark of real neuroscience abstracts, LLMs predicted which result was the true one better than human experts (Luo et al., *Nature Human Behaviour*, 2024).

Models in an afternoon

Experimentalists without a modelling collaborator can get a fitted model, with an explanation, in an afternoon (DePasquale, pre-read 1).

A pipeline in a day

A cognitive-neuroscience lab prototyped a complex decoding pipeline for its own data within a day, then wrote a lab AI policy (Schuck, pre-read 2).

> Which of these would change *your* week first?

POLL 1 OF 6 · WHERE FIRST

Where will agentic coding change research practice most in the next five years? *Pick up to two.*

- **A** Analysing data and building models
- **B** Designing and running experiments: automated labs
- **C** Reading and synthesising the literature
- **D** Writing papers and grants
- **E** Peer review and replication

Zoom poll now · results right after.

Code was friction between a question and an answer

The plumbing stops being the bottleneck

Hours of searching forums and documentation become minutes.

Exploration gets cheap

Five model specifications, three plots, two normalisations, in the time of one.

The barrier drops for the whole lab

The student who "can't code" can build a working pipeline; experimentalists can model.

Scripts over clicks

An analysis written as code can be re-run, reviewed and shared: better for reproducibility.

> ⚠️ It is also trivially easy to produce **plausible · beautiful · wrong**.

"Models at the speed of thought": four directions

1 · Experimentalists model

Fit a model to your own data without waiting months for a modelling collaborator.

2 · Theorists iterate

Try, test and reject new models in hours, at last at the speed theory promised.

3 · Search model space

AI-guided search (AlphaEvolve-style) explores many candidate models at scale.

4 · Maths opens up

Mathematically heavy models become reachable for people stronger at code than at the blackboard.

> *"Major new discoveries are likely to come from humans generating insights, just as they have always done..."* Larry Abbott, Columbia, in his response

"Prolific but shallow"?

> *"The field risks becoming prolific but shallow—generating models faster than we can generate insights."*
>
> Brian DePasquale

- The struggle of turning an idea into equations and code is *"a form of disciplined thinking"*, especially for trainees.
- AI-guided search will also produce *"artifacts of the search process"*.
- His advice: keep stepping away from the AI to exercise your own judgment.

RESPONDENT	THE WORRY, IN THEIR WORDS
Dan Goodman, Imperial	Code that seems to test your idea but hides errors: *"This isn't a theoretical risk; it's something that seems to crop up very regularly in practice."*
Julijana Gjorgjieva, TUM	Expertise is needed *"to recognize when an AI system is inventing concepts that do not actually exist."*
Kim Stachenfeld, DeepMind	LLMs are *"a flashlight as opposed to a laser pointer"*
Anthony Zador, CSHL	*"LLMs are like genies: They can grant you almost any wish, but you have to be very careful about what you ask for."*

Strongest on syntax, weakest on truth

TRUST, BUT SKIM	SLOW DOWN: IT FAILS QUIETLY HERE
Plotting, formatting	**Domain correctness**: units, time zones, scales
Boilerplate, file reading	**Statistics**: the wrong-but-plausible test, leakage
Explaining code	**Your specific method**
Translating languages	**Ground truth**: data meaning, citations

The failure is never a crash, it's a clean run with a wrong number:

- `df.dropna()` silently deletes 40% of rows
- test data leaks into training → a 0.98 that evaporates
- a hallucinated citation nobody checked

And beyond the code: patient data pasted into cloud tools · skills that never develop · a field that becomes *"prolific but shallow"* (DePasquale)

The danger is a confident answer that switches your checking off

- **Automation bias & complacency**: we over-trust automated output and stop checking it *(Parasuraman & Manzey, 2010)*.
- **Cognitive offloading**: the more knowledge workers trust generative AI, the less critical thinking they report *(Lee et al., CHI 2025)*.
- **Illusion of explanatory depth**: fluent output makes you *feel* you understand what you couldn't rebuild *(Rozenblit & Keil, 2002)*.
- **"Cognitive debt"**: heavy LLM use while writing went with weaker memory of one's own text and more homogeneous prose *(Kosmyna et al., 2025, preprint)*.
- **Generation effect**: you understand and retain what *you* produce far better than what you only read *(Slamecka & Graf, 1978)*.

> Offload the typing. **Never offload the understanding** your name depends on.

POLL 2 OF 6 · A REALITY CHECK

In a 2025 randomised trial, experienced open-source developers worked on their own projects with and without AI tools. With AI, they were…

A about 40% faster

B about 20% faster

C no faster and no slower

D about 19% slower

Zoom poll now · the answer is on the next slide.

POLL 2 · THE ANSWER

D: about 19% slower, while feeling 20% faster

+19%
time taken with AI tools, measured

−20%
time the same developers *believed* AI had saved them, afterwards

- 16 experienced developers, 246 real tasks on their own projects, early-2025 tools.
- METR's own caveat: experts in large codebases they knew by heart; it may not hold for small new projects, closer to a hackathon.
- **The lesson is the gap**: your feeling of speed is a poor measurement. That is thinking fast and slow, in one number.

METR (Model Evaluation & Threat Research), "Measuring the impact of early-2025 AI on experienced open-source developer productivity", July 2025.

MODULE 00 · THE THESIS WE'LL REPEAT

You move from author of code to director of code.

Your job becomes intent · judgment · verification. The syntax is the agent's.

Next: how to do that job well.

8 min

17:25 · PART 2 · OBJECTIVE 2 · UNDERSTAND

How vibe coding works

When you write code yourself, understanding comes for free. When you direct an agent, you have to *choose* to engage it. Four disciplines make the new loop safe.

Diverge first, then think critically

1 · Divergence

Science trains **convergence**: narrow to the one right method. Generation is now nearly free, so **diverge first**: *"Give me three approaches, with trade-offs."* Curate rather than produce; converge later, with more on the table.

2 · Critical thinking

The model is confident, fluent and occasionally fabricating, in flawless prose. Treat every output as a pull request from a brilliant, overconfident intern. Ask **"How would I know if this is wrong?"** *before* trusting the pretty plot.

> Trust the vibe to **write** it; never trust the vibe that it's **right**.

Think fast and slow; try and fail

3 · Thinking fast / slow

The AI is your **System 1**: fast, fluent, sometimes wrong. **You** supply System 2: slow, deliberate, checking. *"System 2 is in charge of doubting, but System 2 is often lazy"* (Kahneman).

4 · Try and fail

Generation is cheap → more attempts, smaller stakes. Prototype to throw away. Failure costs **minutes, not a day**, so failure becomes the unit of search, as long as you can always go back (Part 3: git).

> The more the result matters, the more slowly you must read it.

One project folder, one rule file

- Put **code · figures · notes · the paper** in one project folder: the agent sees the whole picture.
- Raw or private **data stays out of what gets shared** (Part 3).
- A rule file at the top, **`AGENTS.md`** (or `CLAUDE.md` for Claude Code), tells the agent your house rules, every time.

> Chat answers a question. A project folder lets the agent **work on your actual project**.

```
my-project/
├── AGENTS.md     ← house rules for the agent
├── NOTES.md      ← what we verified / trusted
├── code/         ← analysis, notebooks
├── results/      ← figures, tables
└── data/         ← stays local, never shared
```

```
# Rules
- Python, pandas. Figures go to results/.
- Never modify files in data/.
- Plan first and wait for my OK.
```

How to ask: direct it like a capable new student

- **Give context, not just the task**: the data, the goal, the constraints, what you've tried.
- **Ask for a plan before code**: *"outline your approach; I'll approve, then you write it."*
- **Constrain it**: language, libraries, *"match the style in this file"*, *"no new dependencies"*.
- **Make it show its work**: *"explain your reasoning; flag anything you assumed."*
- **Small steps**: one change, run, read, correct. Never accept a 300-line dump blind.

Vague in → generic out

❌ *"Analyse my sleep data."*

✅ *"sleep.csv has bedtime and wake time in Geneva local time. Plan how you'd compute sleep duration per night and plot the nightly mean. Don't write code until I approve."*

Three dials, whichever agent you use

DIAL	SETTINGS	USE WHEN
1 · Edit mode: how much rope	**Ask** (touches nothing) · **Plan first** (writes a plan, waits) · **Edit** (you accept each change) · **Auto** (edits and runs on its own)	Exploring → ask · anything big → plan · results that matter → edit · throwaway attempts → auto, **only because git makes it reversible**
2 · Model: the right brain	Frontier / "thinking" models · fast, small models	Statistics, study design, subtle bugs → frontier · boilerplate → fast · stuck → escalate
3 · Steering: hand on the wheel	Interrupt · correct with specifics · paste the error · clear a stale thread	The moment it drifts: a drifting agent rarely fixes itself. Stop it, re-aim, continue

At the hack: your own agent (Claude Code, Codex, GitHub Copilot, Cline), or Cline on open models served on the HPE / NVIDIA environment.

Right mode, right model, hand on the wheel

1. **Plan mode + a strong model** → *"plan how you'd test this; don't write code yet."*
2. Approve the plan → **save a starting point** (Part 3: the agent does it for you).
3. **Edit mode, small steps** → accept good changes, ask *"what changed?"* after each.
4. Stuck or slow? **Switch model** or **interrupt and re-aim**.
5. Mechanical follow-ups (tests, formatting, tidying) → **auto mode**, on a separate branch.

> That's directing.

12 min

17:33 · PART 3 · OBJECTIVE 2 · UNDERSTAND

Git without the jargon

Why an agent needs a safety net, what git does in plain words, and how to hand 95% of it to the agent.

An agent can edit ten files in thirty seconds, confidently, and sometimes wrong

- Without a safety net, a bad edit **silently overwrites** what worked yesterday.
- "Undo" in your editor won't save you across many files, runs and days.
- You need a **known-good state** you can always return to, and a **record** of what changed, when and why.

You already know the problem

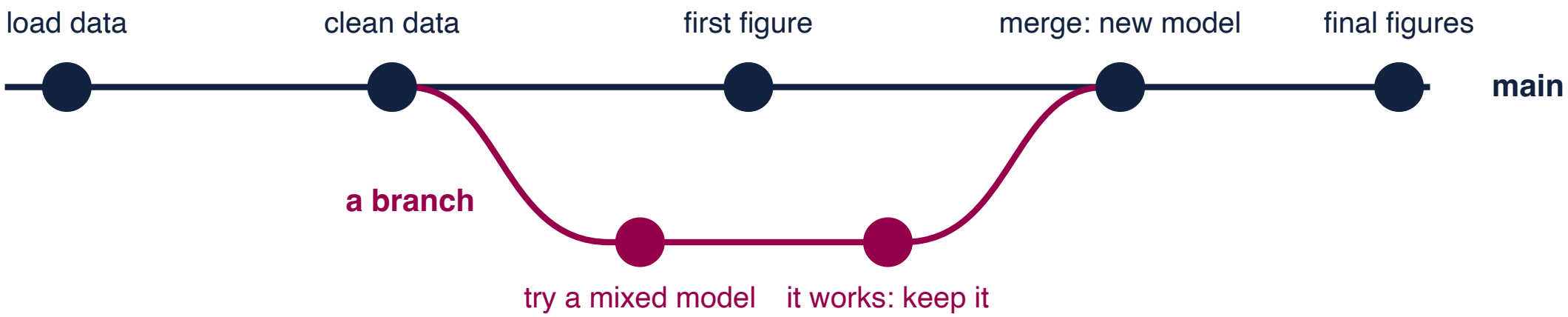
`analysis_v2_final_FINAL_really.py` · `thesis_comments_JM_v3.docx`

Git replaces that with one folder that remembers every saved version.

> Cheap experiments (pillar 4) are only safe if failure is **reversible**. That's git.

THE WHOLE IDEA IN ONE PICTURE

Git is a time machine for a project folder



● Commit = a save point

A snapshot of the whole folder, with a one-line note: *"clean data: removed non-wear days"*.

Branch = a parallel line

Try something risky without touching the version that works.

Merge = bring it back

If the attempt works, fold it into **main**; if not, drop the branch.

THE DICTIONARY · TEN WORDS YOU WILL HEAR

Git and GitHub words, in plain English

WORD	PLAIN MEANING	EVERYDAY ANALOGY
Repository ("repo")	A project folder that remembers its whole history	A lab notebook that never loses a page
Commit	A save point, with a note saying what changed	A dated, signed notebook entry
Diff	What changed since the last save point (removed in red, added in green)	Track changes in Word
Branch · **main**	A parallel line of work · the line that works	A draft copy · the published version
Merge	Fold a branch's changes back into main	Accepting tracked changes
GitHub	A website that hosts a copy of your repo: backup, sharing, review	Google Drive, for projects with history
Push · **pull** · **clone**	Send your save points to GitHub · get others' · download a whole repo	Upload · sync · download
Pull request ("PR")	A page asking "please review these changes and merge them into main"	Sending a draft to your supervisor
`.gitignore`	A list of files git must never save or share (data, passwords)	A "do not photocopy" stamp

Git lives on your laptop; GitHub is the shared copy

Your laptop

The project folder with its save points. Works offline, private, no account needed. **Protects you from yourself.**

push →
← pull

GitHub

The online copy: survives a dead laptop; one place for the team; pull requests for review. **Protects you from the world.**

← pull
push →

Your teammates

Each has their own copy and their own branches; GitHub keeps everyone in sync.

PRIVATE REPOSITORY	PUBLIC REPOSITORY
Only you and invited collaborators	Anyone can read and copy it
Start here while the work is live	At publication, for reproducibility, after checking the history

⚠️ Public is permanent: a password or a dataset pushed in public may be copied before you delete it. Data stays out of git either way.

The vibe-coding loop *is* the git loop

1. **The agent changes something** (you asked it to).
2. **You ask: "what changed?"** It summarises in plain words; the detailed diff is there if the result matters. *This is your System 2 checkpoint.*
3. **Good? "Save it."** A save point with a clear note.
4. **Bad? "Undo that."** Back to the last save point, nothing lost but the bad change.

> **Save before every big request**: a guaranteed way back.

YOU SAY	THE AGENT RUNS
"What did you change?"	`git status` · `git diff`
"Save it: cleaned the units"	`git add` · `git commit -m "…"`
"Undo that"	`git restore .`
"Show me the history"	`git log --oneline`

You don't need to remember the right-hand column. You do need to know it exists, and to read what the agent tells you it did.

Try the risky thing on a branch

- Want the agent to attempt a big refactor, or a second way of analysing the data? Ask for it **on a branch**.
- **main** stays known-good while the agent experiments.
- Bad attempt? **Drop the branch**: zero trace, zero regret. Good? **Merge it.**
- At the hack, each person works on their own branch: `firstname/topic`.

> Branches turn *"I hope this doesn't break everything"* into *"let's just try it."*

YOU SAY	THE AGENT RUNS
"Try a mixed model on a separate branch"	`git switch -c thomas/mixed-model`
"Go back to the main version"	`git switch main`
"It works, keep it"	`git merge thomas/mixed-model`
"Forget it"	`git branch -D thomas/mixed-model`

Hand off 95% of git: say what you want, in plain words

YOU SAY	WHAT THE AGENT DOES FOR YOU
"Start a branch for this task."	Creates `firstname/topic` and explains it in one sentence
"Save my work with a clear message."	Commits, with a note saying what changed and how it was checked
"What changed since this morning?"	Summarises the save points and changes in plain language
"Undo what you just did."	Goes back to the last save point, after telling you what will be lost
"Get my teammates' latest work."	Pulls from GitHub; if two people changed the same lines, explains the conflict and asks which version to keep
"Send it to GitHub and open a pull request."	Pushes the branch, opens the PR, writes what was verified and what was trusted

One-time setup: a GitHub account with two-factor authentication, then `gh auth login` once on your machine. From then on the agent can run `git` and `gh` (GitHub's command-line tool) on your behalf.

THE 5% THAT STAYS YOURS

The agent runs git. You keep five decisions.

When to save

Ask for a save point at each step that works, and before every big request.

Read the summary

Every change gets a plain-language summary; read the detailed diff for anything that reaches a result.

Approve the merge

Nothing reaches **main** without your OK. Never let auto mode push to main unreviewed.

Keep data out

No data files, passwords or keys in git. The agent refuses; `.gitignore` backs it up.

Own the history

Commits carry **your** name. For credit at the hack, the history is the evidence.

> The agent drafts; **you merge**.

ONE FILE MAKES IT WORK · `AGENTS.MD` IN TONIGHT'S PRACTICAL

Write the rules once; the agent follows them every time

- The agent reads `AGENTS.md` at the start of every session (Claude Code via `CLAUDE.md`).
- It tells the agent to **run git for you, explain each step in plain words, and wait for your OK** before anything irreversible.
- Every team repository at the hack ships with one.

```
# Rules for the AI agent working in this folder
You are helping researchers who may not know git.
You run git and GitHub for them; they decide.
Explain every git step in one plain-English sentence.

- Starting a task: create a branch <firstname>/<topic>.
- Saving work: after each step that works, commit with a
  plain-English message. Tell the human "Saved: …".
- "Undo that": go back to the last saved state, after saying
  what will be lost.
- "Send it to GitHub": push and open a pull request saying
  what was verified and what was trusted.
- Never force-push, rewrite history, or merge into main
  without the human's explicit OK.
- Never commit data files, passwords or keys.
```

POLL 3 OF 6 · THE LITERATURE

Soon an agent can re-run any published analysis from its repository in minutes. What happens to your field's literature?

- **A** Replication becomes routine and the literature gets cleaner
- **B** A wave of corrections and retractions, then a cleaner literature
- **C** Little changes: nobody is rewarded for checking
- **D** AI-generated papers multiply faster than anyone can check them

Zoom poll now · results right after.

35 min

17:45 · PART 4 · OBJECTIVE 2 · UNDERSTAND, BY DOING IT

Practical in pairs

Zoom breakout rooms of two. Round 1 on synthetic sleep data, round 2 on a real, open EEG recording. One driver, one skeptic, and the agent runs git for you.

Two roles, switch at 18:06

Driver

Shares their screen and talks to the agent. Says out loud what they are about to ask, and why.

Skeptic

Never touches the keyboard. After every answer: *how would we know if this is wrong?* Decides when to slow down.

YOU HAVE…	SETUP (2 MIN)
VS Code with an agent (Claude Code, Codex, GitHub Copilot, Cline)	Download the practical folder from the link in the chat (or ask your agent to clone it), open it in VS Code, open the agent. It reads `AGENTS.md` and runs git for you
Only a chat AI (ChatGPT, Claude, Gemini…)	Upload the CSV files and READMEs to the chat: tonight's data is synthetic or open, so this is allowed. Skip the git steps
Nothing working yet	Be the skeptic first; in round 2, drive from your partner's machine (Zoom remote control) or in chat mode

ROUND 1 · DRIVER A · 17:50 → 18:06 · `SLEEP-MOOD/`

How long do people sleep each night?

1. *"Brief me on `sleep-mood/` and its codebook. Don't change anything."*
2. *"Start a new branch for this task."* (It explains what a branch is in one sentence.)
3. *"Plan how you'd compute sleep duration per night and plot the average per night. Wait for my OK."*
4. Approve, let it run, **look at the plot together**. Skeptic: is anything strange?
5. If something is: find out **why**, and fix it with a precise instruction.
6. *"Save my work with a clear message, and add a line to NOTES.md."*

The data

Synthetic smartphone and wearable data: 40 participants, 21 nights from 12 October, a four-item mood questionnaire each evening. Codebook: `sleep-mood/data/README.md`.

Pillars to watch

Fast (orient, run) · Diverge (plan) · **Slow** (the plot) · Critical (the fix) · Try and fail (save, or undo)

SWITCH! · ROUND 2 · DRIVER B · 18:06 → 18:20 · `EEG-EYE-STATE/`

Can EEG tell whether the eyes are closed, and how well, honestly?

1. *"Load `eeg-eye-state/data/eeg_eye_state.csv`, describe it, and plot the 14 channels over time."* Skeptic: does that look like EEG?
2. *"Train a simple classifier to predict `eye_closed` and report its accuracy."*
3. Skeptic: *"Would it still work on the last part of the recording if we trained on the first part?"* Ask the agent to test exactly that, and compare.
4. *"Save my work, send it to GitHub and open a pull request saying what we verified and what we trusted."* (No GitHub account yet? Just save.)

The data · real and open

One continuous 117-second recording, Emotiv headset, 14 channels, eyes open or closed from video. Roesler (2013), UCI Machine Learning Repository, CC BY 4.0.

Bring back to the main room

In the chat: (1) the strange thing from round 1 · (2) your two accuracy numbers from round 2.

Five prompts worth stealing

1. *"Explain what this code does before you change anything."*
2. *"Outline your plan, then wait for my OK."*
3. *"Write a test that would fail if this result were wrong."*
4. *"What could make this conclusion wrong? Argue against yourself."*
5. *"Summarise in plain language what you changed and what you assumed."*

> And for git, just say it: *"start a branch"* · *"save my work"* · *"what changed?"* · *"undo that"* · *"send it to GitHub"*.

DEBRIEF · WHAT THE PAIRS FOUND

Two clean runs, two wrong numbers

Round 1 · the night of 24 → 25 October

Mean sleep **5.9 h** subtracting clock times, against **6.9 h** in UTC and on every other night. Clocks went back at 03:00 on Sunday: no error, no warning. Caught by a **plot** and the **calendar**: domain knowledge.

Round 2 · eyes closed from EEG

Four artifact samples (up to 715,897 against ~4,300) flatten every plot. Then: **96%** accuracy with a random split, **46%** when tested on a later part of the recording, below the **55%** of always guessing "open". Neighbouring samples are near-copies.

> Both agents were **mostly right**: the dangerous kind of wrong. What caught them was a skeptic asking one more question.

POLL 4 OF 6 · TRAINING

A first-year PhD student can now get working analysis code from an agent in minutes. In their first year, they should…

- **A** Use agents freely: directing them is the skill that matters now
- **B** Write their core analyses by hand first, agents later
- **C** Use agents, but explain every line at lab meeting
- **D** Keep agents away from thesis-critical analyses

Zoom poll now · results right after.

10 min

18:25 · PART 5 · REVIEW HUNT · EVERYONE, IN THE MAIN ROOM

An agent wrote this. Would you sign it?

1. Open `review-hunt/analysis.py` and the codebook `sleep-mood/data/README.md`.
2. Read the output on the right (or run it).
3. Hunt: the code, the data, the method, the summary.
4. In the chat: every problem, and **what tipped you off**.

The prompt the agent got is at the top of the file. Allowed: plots, the codebook, a second AI, your judgment.

```
Analysed 743 participant-days from 40 participants.

1. Associations with next-day mood (Pearson r)
   sleep duration : r = +0.39
   daily steps    : r = +0.25

2. Low-mood day classifier (random forest,
   30% held-out test set)
   AUC = 0.77

Summary
   Physical activity is the stronger correlate
   of mood (r = +0.25 vs +0.39 for sleep).
   Wearable data flag low-mood days with high
   accuracy (AUC = 0.77), supporting passive
   sensing as an early-warning signal for
   depressive episodes.
```

THE ANSWER KEY

Five problems, and the headline flips

#	PROBLEM	EFFECT	WHAT CATCHES IT
1	`mood_3` reverse-keyed, averaged as is	mood spread halves; r(sleep) +0.45 → +0.39	the codebook
2	Non-wear days kept as 0 steps	fake steps–mood link: r +0.15 → +0.28	a histogram: spike at 0
3	Same participants in train and test	AUC 0.89 → **0.69** on new people	"would it work on a new patient?"
4	Summary contradicts its numbers	"activity is stronger": 0.25 < 0.39	numbers before prose
5	Overclaim	a low-mood *day* is no depressive *episode*	what does the outcome measure?

Fixed: sleep predicts next-day mood (r ≈ +0.45), steps add little (r ≈ +0.15), the classifier generalises modestly (AUC ≈ 0.69). Problem 3 is round 2's EEG trap, again.

When code is cheap, review is the work

Peer review matters more

No AI-generated analysis reaches a paper without a second human reading the reasoning and the checks.

Show how you checked

Short sprints, visible demos: what you built *and how you verified it*. Pair on the prompts.

Share the rules, track provenance

A lab `AGENTS.md`, shared environments; whose prompt, which model, which data version.

> The failure mode is **unreviewed AI code flowing straight into results**.

POLL 5 OF 6 · THE SCARCE SKILL

In 2031, the scarcest skill in a neuroscience lab will be…

- **A** Programming
- **B** Statistics and methods
- **C** Choosing the question worth asking
- **D** Judging whether a result is true
- **E** Collecting good data from real people

Zoom poll now · results right after.

10 min

18:33 · PART 6 · OBJECTIVE 3 · PREPARE

The hack: where we are

What your needs built, the challenges taking shape, what to do this week, and how it runs on 6–7 November, with or without your own tools.

THE NEEDS CAMPAIGN · CLOSED 15 OCTOBER

Thank you: your needs became the hackathon

181
needs in the library when the call closed

30
contributors, from UNIGE, HUG, the Wyss Center, FCBG, Trieste and UNIL

8
draft challenges distilled from them, aiming for 7

What you asked for most

- Domains: **mental health** (60 needs), **neurology** (30)
- Methods: **AI modelling** (54), **data platforms** (46), **smartphone sensing** (37)

Prizes for the best needs: a CHF 500 Synapsy voucher, and a CHF 500 Fondation Pôle Autisme voucher for child and adolescent development. Winners are announced at the finals.

A PREVIEW · DRAFT OF 14 OCTOBER · FINAL LIST ONLINE BY 23 OCTOBER

Eight challenges taking shape

#	CHALLENGE	IN ONE LINE
1	**Between-visit deterioration**	Spot meaningful change from a person's own baseline, without alert fatigue
2	**Physiology you can trust**	Wearable signals usable day to day: quality, stress vs. exercise or coffee, biofeedback
3	**Behaviour annotation from video**	Validated behaviour labels from naturalistic video, models checked against human coders
4	**Clinical speech and language**	Who spoke when, French transcription, speech markers of memory and mood, anonymised voices
5	**Markerless movement, gaze and pupil**	Lab-grade measures from ordinary cameras and phones, checked against motion capture
6	**Linking digital markers to the brain**	EEG synchronised with gaze, speech and movement
7	**Cross-species behaviour and touch**	Mice and humans: pain and pleasure readouts, social scoring
8	**Interactive interventions**	XR, games and toys that engage and measure at the same time

Notice tonight's datasets: sleep, wearables, mood questionnaires, EEG. You have just practised on the kind of data these challenges use.

THIS WEEK · TWO CLICKS THAT DECIDE YOUR WEEKEND

Choose your challenges, then confirm your place

1 · Rank three challenges

From the email *"choose your challenges and confirm your place"*, or at **facmed-hack.sparkboard.com/challenges**: your 1st, 2nd and 3rd choice, each with a short reason (100 words at most).

2 · Confirm and tell us your needs

Diet · accessibility and any special considerations · **Will you sleep on site on Friday night?** Yes, maybe or no: you may spend the night in the building, so bring a sleeping mat and a sleeping bag.

3 · Teams

The organisers assign teams from your rankings and your research; you may decline. Ranking guarantees neither a challenge nor a seat, so do it now.

Your answers are optional, seen by the organisers only, and deleted after the event. Places are limited to about 50–60 people including support staff.

WITH OR WITHOUT YOUR OWN TOOLS

Two ways in, one team repository

You already have the tools

VS Code on your laptop with your own agent: Claude Code or Codex on your own plan, GitHub Copilot, or Cline. Get your team's repository and work as tonight.

You don't, or you'd rather not install

A ready-made VS Code in the browser on the HPE / NVIDIA environment: nothing to install, an agent already set up on open models served there, your team's repository already in place.

> **Either way, every team can build and test on the HPE / NVIDIA environment**: GPUs for heavy jobs, the challenge datasets (which stay there), and HPE engineers on site.

THE WORKFLOW YOU REHEARSED TONIGHT

How your team works together

WHAT HAPPENS	IN PLAIN WORDS
One **repository** per team, in the hack's GitHub organisation	One shared project folder, with its history
Your own **branch** per task: `firstname/topic`	Your own line of work; the version that works stays safe
Save small and often, under your own name	For credit candidates, that history is the evidence
Changes reach main through a **pull request**, with `NOTES.md` updated	A teammate sees what changed, what was verified, what was trusted
Tag `demo` before the finals	The jury looks at exactly what you showed
Data never goes into git; real data stays on the HPE / NVIDIA environment	Laptops get small synthetic samples; agents see structure, never records

Each team repository comes with an `AGENTS.md` like tonight's: your agent already knows these rules.

FINAL PREPARATION

Before 6 November

	WHAT	WHEN
1	**Rank your challenges and confirm your place**, with your needs (diet, access, sleeping on site)	This week
2	A **GitHub account** with two-factor authentication	This week
3	*Optional:* VS Code + one agent; redo tonight's practical on your own	Before 6 Nov
4	**Crash Course #3**: grant writing for pop-up projects, with the UNIGE Research & Grants Office	Thu 29 Oct, 17:00
5	Bring one idea for your challenge, a sleeping mat if you stay, and your curiosity	Fri 6 Nov, Campus Biotech

SATURDAY 7 NOVEMBER · 17:00 → 18:30 · PUBLIC FINALS

32 hours to seed a research project

The goal

A proof of concept or preliminary results that can seed a research project and a grant application, and an inspiring pitch. A working prototype is a bonus.

The audience

Public finals at the CMU, part of the Faculty of Medicine's 150th anniversary, in front of researchers, clinicians and philanthropic funders.

The prizes

Research-aligned: compute credits, equipment and resources. Winning teams are first in line for support after the weekend.

17 min

18:43 · PART 7 · OBJECTIVE 1 · IMAGINE

A future worth building

How a lab, and a field, can govern agentic AI. What becomes possible if we do. And your place in it.

Governance starts in the lab: one lab's AI policy

Learning first

Do by hand the work that builds core skills: questions, models, arguments. Hand off what you don't need to learn.

Draft before AI

"You must always draft a text yourself before handing it to AI."

Verify by falsifying

"AI output sounds confident even when wrong, so know how you can falsify what a model produces." Scripts that check, rather than asking the model.

Limit the risks

"Participant data should not be shared with AI tools, and agents only get access to the folders they need."

Learn the tools

Output quality depends on how well you scope and prompt: invest the time.

You own it

AI is a tool, never a co-author. "The model said so" is no defence: you own everything you make public.

Talk about it openly, then set norms together

> *"Our lab policy's biggest effect wasn't any single rule, but rather creating a culture in which we discuss AI use rather than hide it."*

> *"We should not only have lab policies but decide as a field where we stand on the shifts in money, priorities and agenda that come with AI."*

Nicolas Schuck, University of Hamburg

Five rules for each of us (module 09: your name is on the paper)

1. **Confidentiality**: no patient, unpublished or embargoed data in uncleared tools.
2. **Provenance**: commit the code, pin versions, keep the prompts.
3. **Correctness is yours**: verify anything that reaches a result.
4. **Disclosure**: say how AI was used; AI is no author; check the journal's and funder's policy.
5. **Keep your skills**: enough fluency to review what it writes.

A bright, sustainable, safe future with AI in science

What becomes possible

- **More time for the questions**, and for patients: the plumbing shrinks.
- **Science for more people**: clinicians build their own tools, small labs model like big ones, students contribute from week one.
- **A cleaner literature**: every analysis in a repository, re-runnable by anyone.
- **Open by default**: open models, open data (tonight's EEG), shared code.
- **Faster from need to answer**: a clinical need on Monday, a working prototype by Friday.

What it takes

- **Judgment kept sharp**: you stay the System 2.
- **Norms**, set openly, in labs and as a field.
- **Safe infrastructure**: sensitive data stays where it belongs; the hack's HPE / NVIDIA environment is one example.
- **Sustainable use**: the right-sized model for the task, no compute wasted on throwaway runs.
- **Communities that practise together**: like the one meeting on 6 November.

In the chat: one thing you would build, or fix, in your field if the plumbing were free.

POLL 6 OF 6 · DEPTH

Pre-read 1 warns that AI coding could make theoretical neuroscience *"prolific but shallow"*. For your own field, you think…

- **A** It is already happening
- **B** It will happen unless incentives change
- **C** The opposite: faster coding frees time to think
- **D** Too early to say

Zoom poll now · results right after.

THE ROOM'S ANSWERS · ALL SIX POLLS

What you imagined, and voices to argue with

POLL	THE ROOM	A VOICE FROM THE FIELD
1 · Where first	*majority*	Agentic coding is already removing the engineering bottleneck in modelling (DePasquale)
2 · Reality check	**D** · 19% slower	…while feeling 20% faster (METR, 2025)
3 · Literature	*majority*	Incentives already reward producing models over understanding them (Dan Goodman, Imperial)
4 · Training	*majority*	Trainees do by hand the work that builds core skills (Schuck's lab policy)
5 · Scarce skill	*majority*	LLMs are *"a flashlight as opposed to a laser pointer"* (Kim Stachenfeld, DeepMind)
6 · Depth	*majority*	*"Prolific but shallow—generating models faster than we can generate insights"* (DePasquale)

THE OPENING QUESTION, REVISITED

Where will agentic coding change science?

What we saw tonight

Analysis in minutes · a plan before the code · git run by the agent · two clean runs with wrong numbers, caught by a skeptic · review as the real work

Still imagination

Your answers from 17:00 that nothing tonight showed yet: where your own research, career or field could go

> **Last line in the chat:** *"When agents write the code, the part of my research that stays mine is …"*

SEE YOU ON 6 NOVEMBER

Director, not author.

The agent brings **speed and breadth**. You bring **the question, the judgment and the responsibility**. That division of labour is the skill.

Start this week

One low-stakes task: let an agent into one real analysis; keep the prompt, verify the number (module 09).

Read

DePasquale, *Models at the speed of thought* · Schuck, *How, when and why to use agentic AI in our neuroscience labs* · *The Transmitter*, 2026

The course

github.com/wazaahhh/vibe-coding-crash-course · fork it, star it, open a pull request

Vibe Coding for Scientists · CC BY-NC 4.0 · Thomas Maillart

STILL TRUE AT 19:00

> **"AI is obviously going to have a massive impact on how we do science, and it's hard to know what the long run looks like."**

Anthony Zador · Cold Spring Harbor Laboratory · *The Transmitter*, June 2026

So keep the conversation going: in your lab, and on 6 November.